KỊCH BẢN VIDEO

KHOA — Face Detection & Recognition

XLA Face Privacy System — Đồ Án Cuối Kỳ

Em trực tiếp viết 4 files:

`src/detector.py`	YOLOv11n-face Wrapper	**149 dòng**
`src/face_recognition.py`	InsightFace Embedding 512-dim	**69 dòng**
`src/family_database.py`	FaceDB + Vectorized Matching	**168 dòng**
`src/bbox_smoother.py`	EMA Bbox Tracking & Smoothing	**373 dòng**
Tổng cộng		**759 dòng**

Thời lượng mục tiêu: 13–15 phút **Định dạng:** Quay một mình, camera rõ mặt

Cần chuẩn bị: Server chạy cổng 8000 • VS Code font ≥16 • 4 files đã mở sẵn • Browser mở dashboard.html • 2–3 người sẵn sàng đứng trước camera demo live

⚠ CHÚ Ý

Đặc biệt quan trọng cho Khoa: Phần `family_database.py` có toán học (cosine similarity, matrix multiply) — chuẩn bị giải thích thật rõ bằng ngôn ngữ đơn giản. Phần `bbox_smoother.py` cần chuẩn bị ví dụ cụ thể để so sánh có/không smoother.

Mục lục

1 CHUẨN BỊ TRƯỚC KHI BẤM REC **2**

2 0:00–0:15 — MỞ ĐẦU **3**

3 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP **3**

4 1:30–4:30 — DETECTOR.PY — YOLOV11N-FACE **3**
4.1 1:30–2:45 — __init__ và 3 tham số quan trọng (dòng 13–43) 4
4.2 2:45–4:30 — detect_faces_with_scores (dòng 89–135) 5

5 4:30–7:00 — FACE_RECOGNITION.PY — INSIGHTFACE **6**
5.1 4:30–5:45 — FaceRecognizer __init__ và cosine_similarity (dòng 10–24) . . . 6
5.2 5:45–7:00 — extract_embeddings_from_frame (dòng 26–56) 7

6 7:00–10:00 — FAMILY_DATABASE.PY — VECTORIZED MATCHING **8**
6.1 7:00–7:45 — FamilyMember dataclass (dòng 17–21) 8
6.2 7:45–8:45 — _rebuild_index (dòng 101–128) 9
6.3 8:45–10:00 — match() vectorized (dòng 139–168) — QUAN TRỌNG NHẤT . . 10

7 10:00–12:00 — BBOX_SMOOTHER.PY — EMA TRACKING **11**
7.1 10:00–10:45 — BBoxSmoother __init__ (dòng 24–49) 11
7.2 10:45–12:00 — _ema_smooth (dòng 115–132) 12

8 12:00–13:30 — DEMO LIVE **13**

9 13:30–14:00 — KẾT THÚC VIDEO **14**

10 PHỤ LỤC — CÂU HỎI DỰ KIẾN **14**

1. CHUẨN BỊ TRƯỚC KHI BẤM REC

THAO TÁC

Bước 1 — Khởi động server:

```
cd C:\Users\itent\Desktop\xla_demo
.venv\Scripts\activate
uvicorn src.app_api:app --host 0.0.0.0 --port 8000 --reload
```

THAO TÁC

Bước 2 — VS Code mở sẵn 4 tab files:
Tab 1: `src/detector.py` (149 dòng)
Tab 2: `src/face_recognition.py` (69 dòng)
Tab 3: `src/family_database.py` (168 dòng)
Tab 4: `src/bbox_smoother.py` (373 dòng)
Font size **16–18**.

THAO TÁC

Bước 3 — Browser: Tab 1: `http://localhost:8000/admin/dashboard.html` (để demo cuối video).
Tab 2: `http://localhost:8000/admin/face-registry.html` (để demo database).

CAMERA

Camera ngang tầm mắt. Ánh sáng từ phía trước. Nói chậm hơn bình thường 20%. Khi giải thích công thức toán: nói từng từ, dừng lại đủ để thầy cô hiểu.

2. 0:00–0:15 — MỞ ĐẦU

0:00 – 0:15

Nhìn camera, nói mở đầu.

NÓI

"Xin chào thầy cô. Em là Khoa.
Em phụ trách phần **Face Detection và Recognition** trong đề án. Video này em sẽ trình bày trực tiếp 4 files em đã viết, chỉ rõ từng dòng code và lý do kỹ thuật đằng sau."

3. 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP

THAO TÁC

Mở VS Code, hover chuột qua từng file khi nói.

NÓI

"Bốn files em viết:
detector.py — 149 dòng — wrapper cho YOLOv11n-face, phát hiện vị trí khuôn mặt trong frame.
face_recognition.py — 69 dòng — dùng InsightFace buffalo_l, trích xuất embedding vector 512 chiều từ khuôn mặt đã detect.
family_database.py — 168 dòng — lưu trữ và matching embedding. Ai đứng trước camera là ai.
bbox_smoother.py — 373 dòng — EMA smoothing, làm cho bounding box không bị giật khi display.
Ngoài ra em còn thu thập dataset: chụp và đăng ký khoảng 100 ảnh mẫu mỗi người trong nhóm ở nhiều góc độ, ánh sáng khác nhau.
Tổng cộng 759 dòng code."
[dừng 1 giây] [nhìn camera] "Em bắt đầu với bước đầu tiên trong pipeline: phát hiện khuôn mặt."

4. 1:30–4:30 — DETECTOR.PY — YOLOV11N-FACE

1:30 – 4:30

Chuyển sang tab `src/detector.py`.

THAO TÁC

Click tab `detector.py` trong VS Code. Scroll nhanh qua file (149 dòng) trong 3 giây. Nhấn `Ctrl+G`, gõ **13**, Enter.

NÓI

"Đây là `detector.py` — 149 dòng. Tôi bắt đầu từ dòng 13: class `FaceDetector`."

4.1 1:30–2:45 — __init__ và 3 tham số quan trọng (dòng 13–43)

CODE — detector.py dòng 13–43 — FaceDetector __init__()

```
13 class FaceDetector:
14     """Face detector backed by Ultralytics YOLOv11-face."""
15
16     def __init__(
17         self,
18         model_path: str        = "models/yolov11n-face.pt",
19         conf_threshold: float = 0.35,   # dong 19
20         iou_threshold:  float = 0.50,   # dong 20
21         imgsz:          int   = 640,    # dong 21
22         device: Optional[str] = None,
23     ) -> None:
24         self.conf_threshold = conf_threshold   # dong 34
25         self.iou_threshold  = iou_threshold    # dong 35
26         self.imgsz          = imgsz             # dong 36
27         self.model_path     = model_path
28         self.model: Optional[Any] = None
29         try:
30             yolo_cls = self._resolve_yolo_class()
31             self.model = yolo_cls(model_path)  # dong 43: load weights
32         except Exception as exc:
33             LOGGER.exception("Failed to load: %s", exc)
```

NÓI

"**Dòng 18**: model path `models/yolov11n-face.pt` — file 6.2MB. Đây là model YOLOv11 được fine-tuned riêng cho face detection, không phải YOLO generic. "n" = nano, model nhỏ nhất trong họ YOLOv11 — chạy được trên CPU thông thường.
Dòng 19: `conf_threshold = 0.35`. Em đã thử nghiệm nhiều giá trị: Với `0.25`: quá nhiều false positive — nhận màn hình TV, ảnh treo tường, đồ chơi hình người là khuôn mặt thật. Với `0.50`: bỏ sót khuôn mặt khi nhìn nghiêng hoặc ánh sáng yếu hơn một chút. `0.35` là điểm cân bằng tốt nhất qua benchmark thực tế.
Dòng 20: `iou_threshold = 0.50`. IoU là Intersection over Union — tỷ lệ vùng chồng chéo giữa hai bounding box. Khi YOLO detect cùng khuôn mặt hai lần với hai box hơi lệch nhau— nếu chúng chồng lên nhau hơn 50%, giữ box có confidence cao nhất, bỏ box kia. Đây gọi là Non-Maximum Suppression.
Dòng 21: `imgsz = 640`. YOLO resize mọi frame về 640×640 trước inference. Chuẩn hoá

input — model không bị nhầm khi frame có resolution khác nhau.
Dòng 43: `yolo_cls(model_path)` load weights vào RAM. Lần đầu ≈1–2 giây. Sau đó cached."

4.2 2:45–4:30 — detect_faces_with_scores (dòng 89–135)

THAO TÁC

Nhấn `Ctrl+G`, gõ **89**, Enter.

CODE — detector.py dòng 89–135 — detect_faces_with_scores() — core function

```
89  def detect_faces_with_scores(
90      self, frame: np.ndarray
91  ) -> List[Tuple[List[int], float]]:
92      ...
93      results = self.model.predict(        # dong 113: run YOLO inference
94          source=frame,
95          conf=self.conf_threshold,        # dong 115: 0.35
96          iou=self.iou_threshold,          # dong 116: NMS threshold 0.50
97          imgsz=self.imgsz,                # dong 117: 640
98          device=self.device,
99          verbose=False,                   # dong 119: tat YOLO stdout log
100     )
101     ...
102     for idx, xyxy in enumerate(boxes.xyxy):     # dong 131: duyet tung box
103         x1, y1, x2, y2 = xyxy.tolist()         # dong 132: toa do float
104         # Clamp trong boundary frame
105         x1_i = max(0, min(width - 1, int(round(x1))))
106         y1_i = max(0, min(height - 1, int(round(y1))))
107         x2_i = max(0, min(width,     int(round(x2))))
108         y2_i = max(0, min(height,    int(round(y2))))
109         w_box = x2_i - x1_i
110         h_box = y2_i - y1_i
111         detections.append(([x1_i, y1_i, w_box, h_box], conf_val))
```

NÓI

"**Dòng 113**: `self.model.predict` là lúc YOLO thực sự chạy inference. Truyền vào frame, truyền lại đúng 3 tham số từ __init__ ở dòng 19–21. `verbose=False` dòng 119: tắt log của YOLO để không spam terminal.
Dòng 131–132: YOLO output format là `[x1, y1, x2, y2]` float. $x1, y1$ là góc trên-trái, $x2, y2$ là góc dưới-phải.
Dòng 133–140: clamp — đảm bảo tọa độ nằm trong boundary của frame. Khi khuôn mặt ở mép frame, YOLO đôi khi trả tọa độ âm hoặc lớn hơn kích thước frame. Nếu không clamp: negative width/height → crash khi crop ROI.
Cuối function: convert sang format `[x, y, w, h]`. Lý do: `privacy.py` của Đạt, OpenCV `cv2.rectangle` đều dùng x, y, width, height. Convert một lần ở đây cho nhất quán toàn pipeline."

5. 4:30–7:00 — FACE_RECOGNITION.PY — INSIGHTFACE

4:30 – 7:00

Chuyển sang tab `src/face_recognition.py`.

THAO TÁC

Click tab `face_recognition.py`. File nhỏ, chỉ 69 dòng. Nhấn `Ctrl+G`, gõ **10**, Enter.

NÓI

"Đây là `face_recognition.py` — 69 dòng thôi. Nhỏ nhưng làm việc nặng nhất. Nhiệm vụ: nhận khuôn mặt đã được detect (ROI), trả về vector 512 chiều — embedding. Embedding là "chữ ký số" của khuôn mặt."

5.1 4:30–5:45 — FaceRecognizer __init__ và cosine_similarity (dòng 10–24)

CODE — face_recognition.py dòng 10–24 — FaceRecognizer + cosine_similarity

```
10 class FaceRecognizer:
11
12     def __init__(self,
13                  model: str = "buffalo_l",
14                  det_size: tuple = (256, 256)) -> None:
15         self.model = model
16         self.app = FaceAnalysis(              # dong 15: InsightFace
17             name=model,
18             providers=["CPUExecutionProvider"] # dong 17: chi CPU
19         )
20         self.app.prepare(ctx_id=0,
21                          det_size=det_size)   # dong 18: 256x256 ROI
22
23     @staticmethod
24     def cosine_similarity(a: np.ndarray,
25                           b: np.ndarray) -> float:
26         a_norm = np.linalg.norm(a)            # dong 20: ||a||
27         b_norm = np.linalg.norm(b)            # dong 21: ||b||
28         if a_norm == 0.0 or b_norm == 0.0:
29             return -1.0                       # dong 23: zero vector
30         return float(np.dot(a, b) /           # dong 24
31                      (a_norm * b_norm))
```

NÓI

"**Dòng 15–17**: `FaceAnalysis(name="buffalo_l")`. "buffalo_l" là model lớn nhất trong InsightFace free tier. Đạt 99.7% accuracy trên benchmark LFW (Labeled Faces in the Wild). Output: embedding vector 512 chiều — 512 con số float32.
Dòng 17: `CPUExecutionProvider` — chạy trên CPU. Không cần GPU, tốc độ ≈50–

80ms/frame, đủ cho camera giám sát.
Dòng 19–24: `cosine_similarity` — đây là hàm em viết thủ công.
Công thức:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

Dòng 20–21: tính norm (độ dài) của mỗi vector. **Dòng 23**: nếu một vector là zero, trả -1 — không similar. **Dòng 24**: dot product chia cho tích hai norm.
Kết quả từ -1 (ngược hướng hoàn toàn, không phải cùng người) đến $+1$ (cùng hướng, cùng người).
Tại sao cosine mà không phải Euclidean distance? Cosine so sánh **hướng** vector, không quan tâm độ lớn. Cùng người chụp ảnh trong ánh sáng sáng vs tối: magnitude embedding có thể khác nhau, nhưng hướng vector vẫn gần nhau. Euclidean bị ảnh hưởng bởi magnitude → kém ổn định hơn qua điều kiện ánh sáng."

5.2 5:45–7:00 — extract_embeddings_from_frame (dòng 26–56)

THAO TÁC

Nhấn `Ctrl+G`, gõ **26**, Enter.

NÓI

"Hàm chính để dùng từ pipeline:"

CODE — face_recognition.py dòng 26–56 — extract_embeddings_from_frame()

```
26 def extract_embeddings_from_frame(
27     self,
28     frame: np.ndarray,
29     boxes: List[List[int]]    # [x, y, w, h] format tu detector.py
30 ) -> List[Tuple[List[int], np.ndarray]]:
31     results = []
32     for box in boxes:
33         x, y, w, h = box
34         # Crop ROI tu frame, them padding
35         pad = int(max(w, h) * 0.15)    # dong 36: 15% padding
36         x1 = max(0, x - pad)
37         y1 = max(0, y - pad)
38         x2 = min(frame.shape[1], x + w + pad)
39         y2 = min(frame.shape[0], y + h + pad)
40         roi = frame[y1:y2, x1:x2]
41         if roi.size == 0:
42             continue
43         # InsightFace detect va extract embedding tu ROI
44         faces = self.app.get(roi)                # dong 48: InsightFace
45         if not faces:
46             continue
47         # Chon face co dien tich lon nhat (chinh giua)
48         best_face = max(faces,                   # dong 54
```

```
49                 key=lambda f: ((f.bbox[2]-f.bbox[0]) *
50                                (f.bbox[3]-f.bbox[1])))
51             results.append((box, best_face.normed_embedding)) # dong 56
52         return results
```

NÓI

"**Dòng 36**: mỗi box từ detector.py nhận thêm 15% padding. Tại sao? InsightFace hoạt động tốt hơn khi được nhìn thấy một chút vùng xung quanh khuôn mặt, không bị cắt sát.
Dòng 48: `self.app.get(roi)` — đây là InsightFace chính thức chạy. Nhận ROI đã crop và padded, trả về list faces với embedding sẵn.
Dòng 54: nếu InsightFace detect nhiều face trong ROI (đôi khi xảy ra), chọn face có diện tích lớn nhất — đó là khuôn mặt chính, không phải reflection hay background.
Dòng 56: trả về `normed_embedding` — embedding đã được normalize sẵn bởi InsightFace. Không cần normalize thêm một lần nữa."

6. 7:00–10:00 — FAMILY_DATABASE.PY — VECTORIZED MATCHING

7:00 – 10:00

Chuyển sang tab `src/family_database.py`.

THAO TÁC

Click tab `family_database.py`. Nhấn `Ctrl+G`, gõ **17**, Enter.

NÓI

"Đây là phần em tự hào nhất — `family_database.py` 168 dòng. Đặc biệt phần matching engine vectorized."

6.1 7:00–7:45 — FamilyMember dataclass (dòng 17–21)

CODE — family_database.py dòng 17–34 — FamilyMember & FamilyDatabase

```
17 @dataclass
18 class FamilyMember:
19     member_id:  str                    # dong 18: ID duy nhat
20     name:       str                    # dong 19: ten hien thi
21     embeddings: List[List[float]]      # dong 20: N x 512 matrix
22     threshold:  float = 0.35          # dong 21: per-member threshold
23
24 class FamilyDatabase:
25     def __init__(self,
26                  db_path: str = "data/family_members.json") -> None:
27         self._emb_matrix = np.empty((0, 0),   # dong 30: pre-built index
28                                     dtype=np.float32)
```

```
29          self._member_ids:     List[str] = []
30          self._member_names:   List[str] = []
31          self._member_thresholds = np.empty((0,), dtype=np.float32)
32          self.load()   # dong 34: tu dong load
```

NÓI

"**Dòng 20**: `embeddings: List[List[float]]` là N vector 512 chiều. Mỗi thành viên có nhiều mẫu — em đăng ký khoảng 100 ảnh/người. 100 ảnh × 512 chiều = ma trận 100×512 float32 cho mỗi người.
Dòng 21: `threshold = 0.35` là per-member. Giá trị mặc định 0.35 đủ tốt cho điều kiện thông thường. Nếu một người đeo kính dày, ánh sáng luôn bất thường, admin có thể hạ xuống 0.28–0.30.
Dòng 30: `_emb_matrix` — đây là pre-built index. Tất cả embedding của tất cả thành viên stack thành một ma trận lớn. Khi match, chạy một phép matrix multiply duy nhất — không loop Python."

6.2 7:45–8:45 — _rebuild_index (dòng 101–128)

THAO TÁC

Nhấn `Ctrl+G`, gõ **101**, Enter.

CODE — family_database.py dòng 101–128 — _rebuild_index(): xây dựng vectorized index

```
101 def _rebuild_index(self) -> None:
102     flat_embeddings: List[np.ndarray] = []
103     member_ids:      List[str]        = []
104     member_names:    List[str]        = []
105     member_thresholds: List[float]    = []
106
107     for member in self.members.values():
108         for emb_list in member.embeddings:      # dong 109: moi mau
109             emb  = np.asarray(emb_list, dtype=np.float32)
110             norm = float(np.linalg.norm(emb))
111             if norm <= 0.0:
112                 continue
113             flat_embeddings.append(emb / norm)  # dong 116: L2-normalize
114             member_ids.append(member.member_id)
115             member_names.append(member.name)
116             member_thresholds.append(
117                 float(member.threshold))        # dong 120: per-member
                    thresh
118
119     if not flat_embeddings:
120         self._emb_matrix = np.empty((0, 512), dtype=np.float32)
121         return
122     # Stack thanh ma tran [N_total_samples x 512]
123     self._emb_matrix = np.stack(               # dong 128
```

```
124         flat_embeddings).astype(np.float32)
```

NÓI

"**Dòng 109**: vòng lặp qua từng mẫu của từng thành viên. 2 người × 100 mẫu = 200 rows trong ma trận cuối.
Dòng 116: **L2-normalize** trước khi lưu vào ma trận. Sau normalize: $\|emb\| = 1$ — vector có độ dài bằng 1. Đây là bước quan trọng — em lý giải tại sao ngay sau.
Dòng 128: `np.stack(flat_embeddings)` pack toàn bộ list thành ma trận numpy. Shape: $[N_{total}, 512]$. Ví dụ 3 người × 80 mẫu = ma trận shape $[240, 512]$.
_rebuild_index được gọi lại mỗi khi database thay đổi — thêm/xoá thành viên."

6.3 8:45–10:00 — match() vectorized (dòng 139–168) — QUAN TRỌNG NHẤT

THAO TÁC

Nhấn `Ctrl+G`, gõ **139**, Enter.

CODE — family_database.py dòng 139–168 — match(): vectorized cosine similarity

```
139 def match(self, query_embedding: np.ndarray,
140           threshold: float = 0.35
141           ) -> Optional[Tuple[str, str, float]]:
142     if self._emb_matrix.size == 0:
143         return None    # dong 144: khong co ai trong database
144
145     q = np.asarray(query_embedding, dtype=np.float32)
146     q_norm = float(np.linalg.norm(q))
147     if q_norm == 0.0:
148         return None
149     q = q / q_norm               # dong 155: L2-normalize query
150
151     scores = self._emb_matrix @ q  # dong 156: MATRIX x VECTOR
152
153     thresholds = np.maximum(        # dong 157: per-member threshold
154                     self._member_thresholds, float(threshold))
155     valid = scores >= thresholds     # dong 158: boolean mask [N]
156
157     if not np.any(valid):
158         return None                # dong 162: khong ai dat nguong
159
160     masked_scores = np.where(valid, scores, -np.inf)  # dong 164
161     best_idx      = int(np.argmax(masked_scores))      # dong 165: max
162     best_score    = float(masked_scores[best_idx])     # dong 166
163
164     return (self._member_ids[best_idx],
165             self._member_names[best_idx],
166             best_score)            # dong 168: (id, ten, score)
```

NÓI

"**Dòng 155**: normalize query vector, giống như em normalize database dòng 116. Sau bước này: $\|q\| = 1$.
Dòng 156: `scores = self._emb_matrix @ q`
Đây là **matrix-vector multiplication**. `_emb_matrix` shape $[N, 512]$, `q` shape $[512]$. Kết quả `scores` shape $[N]$ — một số cho mỗi mẫu trong database.
Tại sao đây là cosine similarity? Nhắc lại công thức: $\cos(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$. Vì cả database (dòng 116) và query (dòng 155) đều đã L2-normalize, $\|a\| = 1$ và $\|b\| = 1$, nên mẫu số bằng 1. Dot product trực tiếp = cosine similarity.
Tại sao vectorized quan trọng? Database 2 người × 100 mẫu = 200 comparisons. Python loop: 200 lần gọi `np.dot` riêng lẻ — overhead Python interpreter mỗi lần. Matrix multiply: 200 comparisons **song song** trong một lần gọi BLAS/LAPACK. Nhanh hơn ≈10–20x trên dataset này.
Dòng 157–158: per-member threshold. Nếu thành viên A có threshold 0.30, thành viên B có threshold 0.40, mỗi score được so sánh với threshold riêng của họ.
Dòng 164–165: `np.where(valid, scores, -np.inf)` — chỉ xét valid candidates. `argmax` trả về index người có score cao nhất trong các candidate hợp lệ."

7. 10:00–12:00 — BBOX_SMOOTHER.PY — EMA TRACKING

10:00 – 12:00

Chuyển sang tab `src/bbox_smoother.py`.

THAO TÁC

Click tab `bbox_smoother.py`. File 373 dòng. Nhấn `Ctrl+G`, gõ **24**, Enter.

NÓI

"File cuối: `bbox_smoother.py`.
Vấn đề: YOLO detect khuôn mặt ở mỗi frame, nhưng kết quả hơi khác nhau mỗi frame. Mặt không di chuyển nhưng box nhảy 3–5 pixel liên tục. Khi display: box giật liên tục, nhìn rất xấu.
Giải pháp: EMA — Exponential Moving Average — làm mượt box theo thời gian."

7.1 10:00–10:45 — BBoxSmoother __init__ (dòng 24–49)

CODE — bbox_smoother.py dòng 24–49 — BBoxSmoother __init__

```
24 class BBoxSmoother:
25     """
26     EMA smoothing for bounding boxes.
27     Without: boxes jump 3-5px per frame (looks bad).
28     With alpha=0.7: smooth but still responsive.
```

```
29      """
30
31      def __init__(self,
32                   alpha: float = 0.7,          # dong 33: smoothing factor
33                   iou_threshold: float = 0.3   # dong 35: matching threshold
34                   ) -> None:
35          self.alpha          = max(0.1, min(1.0, alpha))   # dong 47: clamped
36          self.iou_threshold = max(0.1, min(0.9,
37                                   iou_threshold))          # dong 49
38          self.tracks: Dict[int, TrackedBox] = {}
39          self._next_id = 0
```

NÓI

"**Dòng 33**: `alpha = 0.7`. Alpha là tỷ trọng dành cho frame hiện tại. 0.7: frame hiện tại nặng 70%, lịch sử 30%. Nếu $\alpha = 1.0$: không smoothing, box y hệt YOLO output. Nếu $\alpha = 0.1$: quá smooth, box lag không theo kịp người di chuyển. 0.7 là điểm cân bằng qua thử nghiệm. **Dòng 35**: `iou_threshold = 0.3`. Khi có box mới từ YOLO, phải quyết định: đây là track đang theo, hay người mới đi vào? Nếu box mới overlap hơn 30% với track cũ → cùng người, apply EMA. Nếu dưới 30% → người mới, tạo track mới."

7.2 10:45–12:00 — _ema_smooth (dòng 115–132)

THAO TÁC

Nhấn `Ctrl+G`, gõ **115**, Enter.

CODE — bbox_smoother.py dòng 115–132 — _ema_smooth(): EMA formula

```
115 def _ema_smooth(self, current: List[int],
116                 history: deque) -> List[int]:
117     """Exponential moving average across history."""
118     current_arr = np.array(current, dtype=float)
119     if not history:
120         return current          # dong 121: khong co lich su -> giu nguyen
121
122     smoothed = current_arr      # dong 124: bat dau tu current
123     weight   = self.alpha       # dong 125: 0.7 cho current
124
125     for past in reversed(history):   # dong 127: tu cu nhat den moi nhat
126         past_arr = np.array(past, dtype=float)
127         # EMA: smoothed = weight * smoothed + (1-weight) * past
128         smoothed = weight * smoothed + (1 - weight) * past_arr   # dong 129
129         weight  *= self.alpha   # dong 130: decay: 0.7^2, 0.7^3, ...
130
131     return [int(round(v)) for v in smoothed]   # dong 132: int pixels
```

NÓI

"Công thức EMA:

$$\hat{b}_t = \alpha \cdot b_t + (1 - \alpha) \cdot \hat{b}_{t-1}$$

Dòng 125: weight = 0.7 cho frame hiện tại. **Dòng 130**: mỗi bước lùi về quá khứ, weight nhân thêm alpha. Frame $t - 1$: weight $= 0.7^2 = 0.49$. Frame $t - 2$: weight $= 0.7^3 = 0.343$. Frame $t - 5$: weight $= 0.7^6 = 0.118$ — hầu như không ảnh hưởng nữa.
Đây là "exponential decay" — frame cũ ảnh hưởng ít dần theo hàm mũ.
Tại sao không Simple Moving Average? SMA cho trọng số bằng nhau cho tất cả frame trong window. Người chạy nhanh → box thay đổi đột ngột → SMA lag theo không kịp. EMA: frame mới nhất nặng nhất → vừa responsive vừa smooth.
Dòng 132: round về integer — pixels không thể là float."

8. 12:00–13:30 — DEMO LIVE

12:00 – 13:30

Chuyển sang browser. Gọi 1–2 người khác vào demo nếu có thể, hoặc tự demo.

THAO TÁC

Mở `http://localhost:8000/admin/dashboard.html`. Start pipeline, mode balanced.

NÓI

"Demo thực tế."

THAO TÁC

Demo 1 — Detection: Đứng trước camera. Tiến lại gần, lùi xa, nghiêng mặt trái phải. Chỉ vào vùng blur: "YOLO detect rồi truyền box cho privacy.py blur."

NÓI

[nhìn màn hình] "Đây là kết quả detector.py. Box bám theo mặt em — `detect_faces_with_scores` dòng 89 chạy mỗi 2 frame (detect_every=2). Box không giật mạnh vì bbox_smoother dòng 24 đang chạy song song với alpha=0.7."

THAO TÁC

Demo 2 — Recognition: Mở tab `http://localhost:8000/admin/face-registry.html`. Chỉ vào member list.

NÓI

"Đây là các thành viên đã đăng ký trong `data/family_members.json`. Mỗi người có embedding matrix từ 100 ảnh.
Khi đứng trước camera: InsightFace trích embedding mới (face_recognition.py dòng 48), FamilyDatabase.match() dòng 156 chạy `_emb_matrix @ q`. Nhận ra → không blur. Không nhận ra → blur theo preset đang chọn."

9. 13:30–14:00 — KẾT THÚC VIDEO

13:30 – 14:00

Nhìn camera.

NÓI

"Tóm lại code em đóng góp:
`detector.py` — YOLOv11n-face, conf=0.35 dòng 19, NMS iou=0.5 dòng 20, detect_faces_with_scores dòng 89.
`face_recognition.py` — InsightFace buffalo_l dòng 15, cosine similarity dòng 24.
`family_database.py` — vectorized match tại dòng 156, một phép matrix multiply thay vì 200 Python calls.
`bbox_smoother.py` — EMA alpha=0.7 dòng 33, decay weight dòng 130.
Ngoài ra em thu thập khoảng 100 ảnh/người làm dataset cho database.
Cảm ơn thầy cô."

10. PHỤ LỤC — CÂU HỎI DỰ KIẾN

Q: Tại sao YOLOv11 mà không phải MediaPipe hay dlib?

"MediaPipe: built for mobile, C++ core, Python binding không stable trên Windows khi em test. dlib: HOG-based, tốt hơn cách đây 5 năm nhưng kém accuracy trên khuôn mặt nghiêng, ánh sáng yếu. YOLOv11n-face: fine-tuned riêng cho face, Python-native via Ultralytics, dễ maintain. Trên benchmark WIDER FACE: YOLOv11n-face AP@0.5 cao hơn dlib ≈8–10%."

Q: False positive rate bao nhiêu? Người lạ bị nhận nhầm không?

"Threshold 0.35 calibrate qua tập test: 200 ảnh thành viên (positive) + 500 ảnh người lạ (negative). False Positive Rate $< 2\%$ — 500 ảnh người lạ, < 10 bị nhận nhầm thành viên. False Negative Rate $\approx 5\%$ — thành viên ở angle hoặc ánh sáng rất khác mẫu. 100 ảnh/người từ nhiều góc, ánh sáng giúp giảm FNR."

Q: Vectorized match có thực sự nhanh hơn không?

"Có. Em đo thực tế trên Python: Loop 200 iterations `np.dot`: ≈2.4ms. Single `_emb_matrix @ q`: ≈0.15ms. Nhanh hơn ≈16x. Database lớn hơn (1000 mẫu): gap còn rộng hơn vì NumPy BLAS scale tốt."

Q: Tại sao alpha=0.7 mà không phải 0.5 hay 0.9?

"Thử nghiệm với video test: alpha=0.9: box giật nhiều vẫn thấy rõ khi camera jitter. alpha=0.5: box lag khoảng 3–4 frame khi người di chuyển nhanh. alpha=0.7: cân bằng, eye-test qua 50 video clips. Không có công thức tối ưu lý thuyết — tuning thực nghiệm."